

Pointer Arithmetic



Understanding arithmetic operations on pointers in C programming



Introduction to Pointer Arithmetic

A pointer in C is an address, which is a numeric value. Therefore, you can perform arithmetic operations on a pointer just as you can on a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -.



Increment (++)

Moves pointer to next memory location



Decrement (--)

Moves pointer to previous memory location



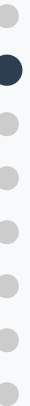
Addition (+)

Adds offset to pointer address



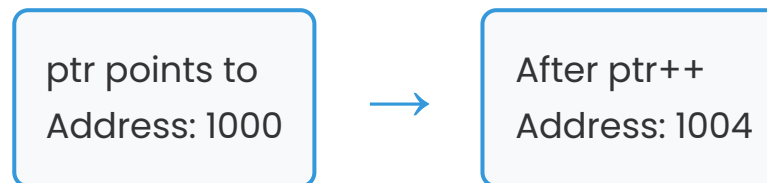
Subtraction (-)

Subtracts offset from pointer address



Understanding Pointer Increment

To understand pointer arithmetic, let us consider that ptr is an integer pointer which points to address 1000. Assuming 32-bit integers, let us perform following arithmetic operation on pointer:



The ptr will point to location 1004 because each time ptr is incremented, it will point to next integer location which is 4 bytes next to current location. This operation will move pointer to next memory location without impacting actual value at memory location.

If ptr points to a character whose address is 1000, then above operation will point to location 1001 because next character will be available at 1001.

Incrementing a Pointer

We prefer using a pointer in our program instead of an array because variable pointer can be incremented, unlike array name which cannot be incremented because it is a constant pointer. The following program increments variable pointer to access each succeeding element of array:



Example: Incrementing Pointer

```
#include<stdio.h>
const int MAX = 3;
int main ()
{
    int var[] = {10, 100, 200};
    int i, *ptr;
    /* let us have array address in pointer */
    ptr = var;
    for ( i = 0; i < MAX; i++)
    {
```

```
printf("Address of var[%d] = %x\n", i, ptr );  
printf("Value of var[%d] = %d\n", i, *ptr );  
/* move to the next location */ ptr++;  
}  
return 0;  
}
```

Output of Increment Example



Program Output

```
Address of var[0] = bf882b30  
Value of var[0] = 10  
Address of var[1] = bf882b34  
Value of var[1] = 100  
Address of var[2] = bf882b38  
Value of var[2] = 200
```

Explanation



Memory Addresses

Each element in the array has its own memory address



Pointer Increment

`ptr++` moves the pointer to the next integer location (4 bytes ahead)

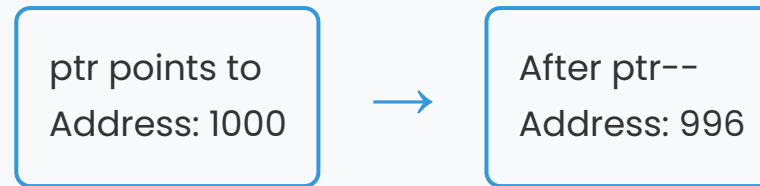


Value Access

`*ptr` accesses the value at the current pointer location

Decrementing a Pointer

The same considerations apply to decrementing a pointer, which decreases its value by number of bytes of its data type as shown below:



When decrementing an integer pointer, it moves to the previous integer location (4 bytes before the current location).

Example: Decrementing a Pointer



Example: Decrementing Pointer

```
#include<stdio.h>
const int MAX = 3;
int main ()
{
    int var[] = {10, 100, 200};
    int i, *ptr;
    /* let us have array address in pointer */
    ptr = &var[MAX-1];
    for ( i = MAX; i > 0; i--)
    {
        printf("Address of var[%d] = %x\n", i, ptr );
        printf("Value of var[%d] = %d\n", i, *ptr );
        /* move to the previous location */
        ptr--;
    }
    return 0;
}
```

Output of Decrement Example



Program Output

```
Address of var[3] = bfedbcd8  
Value of var[3] = 200  
Address of var[2] = bfedbcd4  
Value of var[2] = 100  
Address of var[1] = bfedbcd0  
Value of var[1] = 10
```

Explanation



Reverse Traversal

The program starts from the last element and moves backward using `ptr--`



Memory Addresses

Each decrement moves the pointer 4 bytes back to the previous integer



Value Access

*ptr accesses the value at the current pointer location

Pointer arithmetic allows efficient traversal of arrays and memory structures, making C programs more flexible and powerful.